

Root-cause discovery for this generic error, the key is to treat "Transaction could not be completed" as a **symptom**, then progressively inspect transaction, account, beneficiary, OTP, network, fraud, and backend/switch signals.

Discovery flow

Priority	Check	Discovery
1	Transaction amount vs. daily transfer usage/limit	Daily limit exceeded
2	Available balance vs. ₹50,000 + applicable charges	Insufficient balance
3	Bank/service health at transaction timestamp	Bank server unavailable
4	OTP issuance time vs. verification time	OTP expired
5	Request/response timestamps and gateway logs	Network timeout
6	Beneficiary status and account eligibility	Beneficiary inactive
7	Fraud/risk-engine decision and rule ID	Fraud rule triggered
8	Core-banking dependency health/status	Core banking unavailable
9	UPI switch health and response codes	UPI switch unavailable

Recommended diagnostic sequence

Transaction failed

```
|
|
| └─ Was the request received?
|   └─ No → Network timeout / client-gateway issue
|
|
| └─ Was authentication successful?
|   └─ No → OTP expired
|
|
| └─ Is beneficiary eligible?
|   └─ No → Beneficiary inactive
|
|
| └─ Is ₹50,000 within applicable limits?
|   └─ No → Daily limit exceeded
|
|
| └─ Is sufficient balance available?
```

- | └─ No → Insufficient balance
- |
- | └─ Did fraud/risk checks reject it?
- | └─ Yes → Fraud rule triggered
- |
- | └─ Is core banking responding?
- | └─ No → Core banking unavailable
- |
- | └─ Is UPI switch responding?
- | └─ No → UPI switch unavailable
- |
- | └─ Is the bank/payment service healthy?
- | └─ No → Bank server unavailable

One important design point: **don't infer the cause solely from the customer-facing error text.** "Transaction could not be completed" is intentionally generic; the diagnostic system should correlate the transaction ID with internal status codes, timestamps, dependency health, limits, account state, and risk decisions to identify which of the nine causes actually occurred.